



https://github.com/SETAPESU26/22_sliding_puzzle

Got it. You've given me the **assigned GitHub repo**:

[22_sliding_puzzle GitHub repository](#) ↗

And I have the **Lab 4 handout**. The important requirement is to work on the assigned puzzle repo, first understand its README, run the existing Python game, record the before video, fix broken code, then add **every feature listed in that repo's README one by one**, with separate commits, and finally record the after video and submit the code/videos/chat history under **Lab-4**.



Lab_4_VibeCoding_Student_handout

If you want, I can now go through this GitHub repo **step by step with you**, starting with:

1. What the repo contains
2. What is broken
3. What features the README asks you to add
4. Exact prompts to give ChatGPT/Vibe Coding
5. What code to change
6. How to test each task
7. What to commit after each task
8. How to arrange your **Lab-4** folder and final submission

this is my sliding puzzle project i got for se lab 4. first tell me what this project is doing and explain the files simply. i also have to make some changes given in readme

Yes. I checked the **actual repository files and README**, so I'll explain your project based on what is really there, not guesswork.

1. What is this project?

This is a **terminal-based Sliding Puzzle game** written in Python.

The idea is simple:

- You have a board containing numbered tiles and **one blank space**.
- You move tiles into the blank space.
- You use:
 - **W** → move tile into blank from above
 - **A** → move tile into blank from the right

- S → move tile into blank from below
 - D → move tile into blank from the left
 - Q → quit
- The goal is to arrange the board into:

Plain text

```
1  2  3  4
5  6  7  8
9 10 11 12
13 14 15
```

with the blank at the end.

The current project starts with a **4×4 puzzle**. The README says it should eventually support **3×3, 4×4 and 5×5**. [GitHub](#)

2. Files in your project

Your repo has only 5 files:

Plain text

```
22_sliding_puzzle/
|
├─ README.md
├─ main.py
├─ game.py
├─ puzzle.py
└─ requirements.txt
```

The repository itself confirms these files. [GitHub](#)

Think of them like this:

Plain text

```
main.py
  ↓
Starts the game
  ↓
game.py
  ↓
Controls the gameplay
  ↓
```

puzzle.py
↓
Controls the puzzle/board

And:

</> Plain text

requirements.txt
↓
Python dependency information

README.md

</> Plain text

↓
Tells you what you have to fix/add

3. main.py — starts the game

Your main.py is extremely small:

</> Python

```
from game import SlidingPuzzle

if __name__ == "__main__":
    SlidingPuzzle().run()
` `` :chatgpt-content-reference{index="2"}
```

Simple meaning

This file basically says:

> "Start the Sliding Puzzle game."

It imports the `SlidingPuzzle` class from `game.py` and runs it.

So when you type:

```
` `` bash
python main.py
```

the program starts.

Remember for viva:

main.py = entry point of the project.

4. `game.py` — controls the actual game

This is the main gameplay file.

It handles:

- displaying the board
- counting moves
- timer
- taking user input
- checking whether the puzzle is solved
- quitting the game

The current code creates a 4×4 puzzle:

`</>` Python

```
self.size = 4
self.puzzle = Puzzle(self.size)
self.moves = 0
self.started = time.monotonic()
``` :chatgpt-content-reference{index="3"}
```

### In simple words

When the game starts:

```
```text
Create 4x4 puzzle
    ↓
Set moves = 0
    ↓
Start timer
    ↓
Show board
    ↓
```

It also checks whether the puzzle is solved

The code does:

`</>` Python

```
if self.puzzle.solved():
    print("Solved!")
    return
```

So once the puzzle reaches the solved arrangement, the game exits. [GitHub](#)

5. `puzzle.py` — the actual puzzle logic

This is probably the most important file for your Lab 4 changes.

It handles:

- creating the board
 - finding the blank
 - moving tiles
 - checking if solved
-

How the board is created

The current code does:

`</>` Python

```
tiles = list(range(1, self.size * self.size)) + [0]
random.shuffle(tiles)
``` :chatgpt-content-reference{index="5"}
```

For a 4×4 board, this creates:

```
```text
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

Then `random.shuffle()` randomly mixes everything.

For example:

`</>` Plain text

```

5  1  9  3
2  0  7 14
11 6 15 4
8 13 10 12

```

BUT HERE IS THE BIG PROBLEM

Not every random arrangement of a sliding puzzle is actually solvable.

The README intentionally identifies this as the defect:

`random.shuffle(tiles)` creates an intentional solvability defect. [🌐 GitHub](#)

That's why **Task 1** asks you to fix the starting-state generation.

6. What is `0` ?

In this project:

`</>` Python

```
0
```

represents the **blank/empty space**.

For example:

`</>` Plain text

```

1  2  3
4  5  6
7  0  8

```

Here:

`</>` Plain text

```
0 = blank
```

You can move `8` into the blank.

Result:

Plain text

```
1 2 3
4 5 6
7 8 0
```

7. blank_pos() — finds the blank

In puzzle.py :

Python

```
def blank_pos(self):
    for r in range(self.size):
        for c in range(self.size):
            if self.board[r][c] == 0:
                return r, c
`:`chatgpt-content-reference{index="7"}
```

Simple meaning:

> Search the board until you find `0`.

For example:

```
`:`text
1 2 3
4 0 5
6 7 8
```

It finds:

Plain text

```
row = 1
column = 1
```

So the program knows where the blank is.

8. move() — moves a tile

This function handles W/A/S/D.

It converts the keys into directions:

`</>` Python

```
"w": (-1, 0)
"s": (1, 0)
"a": (0, -1)
"d": (0, 1)
``` :chatgpt-content-reference{index="8"}
```

Basically:

```
```text
W → up
S → down
A → left
D → right
```

Then it checks whether the destination is inside the board.

If it is valid:

`</>` Python

```
self.board[r][c], self.board[nr][nc] = \
    self.board[nr][nc], self.board[r][c]
```

It swaps the blank and the tile.

9. solved() — checks the final arrangement

The project considers the puzzle solved when the board becomes:

`</>` Plain text

```
1 2 3 ... n
...
0
```

The function checks exactly that. [GitHub](#)

For 3×3:

Plain text

```
1 2 3
4 5 6
7 8 0
```

For 4×4:

Plain text

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

10. requirements.txt

This one is easy.

It says the project uses **only Python's standard library** and does not need third-party packages. It recommends Python 3.9+. [GitHub](#)

So you **don't need to install things like pandas, pygame, numpy, etc.**

Basically:

Plain text

```
requirements.txt
↓
No external packages
```

11. The important part — what YOU have to change

Your README gives 4 tasks.

Task 1 — Make every starting puzzle solvable

Current problem:

Python

```
random.shuffle(tiles)
```

can create an impossible puzzle.

You need to change the initialization so that the board is **always reachable from the solved board**.

The README specifically suggests:

</> Plain text

Start with solved board

↓

Make legal blank moves

↓

Scramble it

↓

Guaranteed solvable

```
``` :chatgpt-content-reference{index="11"}
```

---

#### ### Task 2 – Complete puzzle lifecycle

You need to properly handle the situation when the puzzle is solved.

The requirements are:

- detect solved arrangement
- end the game cleanly

So it's only 4×4.

You need:

#### </> Plain text

3 × 3

4 × 4

5 × 5

And moves + timer should work correctly for all three sizes.  GitHub

---

## Task 4 — Correct invalid-move feedback

This is an important one.

Suppose the blank is here:

</> Plain text

```
1 2 3
4 5 0
6 7 8
```

If you try to move right:

</> Plain text

```
D
```

there is nothing to the right.

Therefore:

</> Plain text

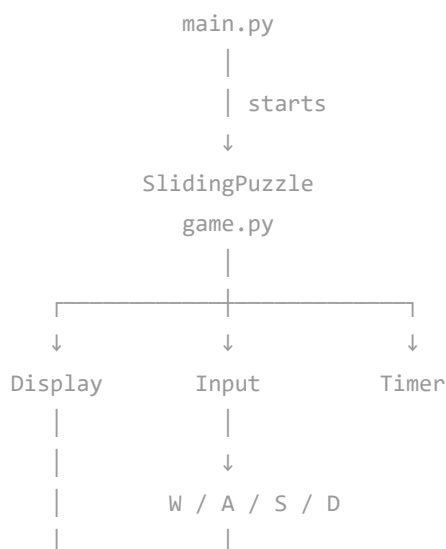
```
NO tile moved
NO move count increase
NO successful-slide message
```

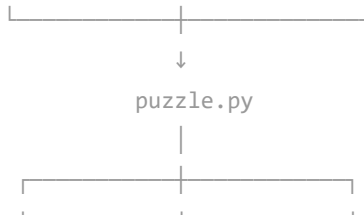
The README explicitly says invalid moves must not increment the move count or generate success feedback. [GitHub](#)

## 12. Your whole project in one diagram

This is the easiest way to remember it for your SE lab/viva:

</> Plain text





## 13. What your professor is actually testing

This lab isn't just asking you to make a game.

The README says the purpose is to use an **LLM as a coding assistant**, while you remain responsible for understanding and testing the result. It specifically says to inspect the existing code first, ask for explanations when needed, test generated code, and keep the complete LLM chat history.  [GitHub](#)

So **don't immediately ask an AI to rewrite the whole project**.

Your workflow should be:

### Plain text

```
graph TD; A[Understand existing project] --> B[Run original project]; B --> C[Record BEFORE video]; C --> D[Reproduce the bug]; D --> E[Prompt AI for Task 1]; E --> F[Understand + test]; F --> G[Commit Task 1]; G --> H[Task 2]; H --> I[Commit Task 2]; I --> J[Task 3]; J --> K[]
```

The README also explicitly says **not to replace the whole project with an unrelated implementation**, not to add persistence, and to keep the code modular.  [GitHub](#)

## One very important thing

**Don't make any changes yet.**

First, we should run the original project exactly as it is and reproduce the **Task 1 broken behavior**. That gives you the correct **10-second BEFORE video** and helps you understand what you're fixing.

Then I'll help you do **Task 1 → Task 2 → Task 3 → Task 4 one at a time**, including the exact natural-looking prompts you can use for your Lab 4 chat history.

task 1 says every generated puzzle should be solvable. in my puzzle.py it is using random.shuffle(). how do i fix this without changing the whole project

Yes — **you do not need to change the whole project**.

The simplest fix for Task 1 is to change **only the puzzle generation inside puzzle.py**.

## What is wrong now?

You currently have something like:

`</>` Python

```
tiles = list(range(1, self.size * self.size)) + [0]
random.shuffle(tiles)
```

`random.shuffle()` randomly arranges the tiles, but some arrangements of a sliding puzzle are **unsolvable**.

## Better approach for your lab

Instead of randomly shuffling all the tiles, start from the **already-solved board** and perform a number of **legal moves** randomly.

For example:

`</>` Plain text

Solved:

```
1 2 3
4 5 6
7 8 0
```

↓ random legal moves

```
1 2 3
4 0 6
7 5 8
```

↓ more legal moves

```

1 2 3
7 4 6
0 5 8

```

Because every position was reached through a legal move from the solved state, it is **guaranteed to be solvable**.

This also matches the README's suggested idea: start from the solved board and scramble it using legal blank moves.

## What you should change

You **keep**:

- `Puzzle` class
- `move()` function
- `blank_pos()`
- `solved()`
- the rest of `game.py`
- the existing project structure

You only change the **initial board generation** in `puzzle.py`.

A clean implementation would be along these lines:

`</>` Python

```

def _scramble(self):
 moves = self.size * self.size * 20

 for _ in range(moves):
 r, c = self.blank_pos()

 possible = []

 if r > 0:
 possible.append("w")
 if r < self.size - 1:
 possible.append("s")
 if c > 0:
 possible.append("a")
 if c < self.size - 1:
 possible.append("d")

 self.move(random.choice(possible))

```

Then instead of:

`</>` Python

```
tiles = list(range(1, self.size * self.size)) + [0]
random.shuffle(tiles)
```

you initialize the board in solved order and call `_scramble()` .

**Important:** Before you replace anything, send me your current `puzzle.py` code (or upload the repo/file). I can then tell you **exactly which lines to change**, without accidentally breaking your existing project.

i changed the code for task 1. how do i test whether the puzzle is actually solvable

Yes. Since you changed Task 1, **don't just assume it works**. Test it by generating the puzzle many times and checking that you can solve it.

## 1. First run your normal game

In the project folder:

`</>` Bash

```
python main.py
```

Generate a puzzle and look at the board.

For example:

`</>` Plain text

```
5 1 3 4
2 0 7 8
9 6 11 12
13 10 14 15
```

Try playing it normally with `W/A/S/D` .

If you eventually reach:

`</>` Plain text

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

then that particular puzzle is solvable.

---

## 2. Better test: generate many puzzles

Because Task 1 says **every generated puzzle should be solvable**, test your program multiple times.

Run:

 **Bash**

```
python main.py
```

Then:

1. Note the starting board.
2. Try solving it.
3. Quit.
4. Run it again.
5. Repeat around **10–20 times**.

You should get different starting arrangements, but every one should be possible to solve.

---

## 3. There is an easier test for YOUR implementation

If you used the approach we discussed — **start from the solved board and make random legal moves** — then the important thing is:

Every generated board is automatically reachable from the solved board.

So you don't actually need to calculate inversion parity separately.

Your logic is:

 **Plain text**

```
Solved board
↓
legal random move
↓
legal random move
↓
legal random move
↓
generated puzzle
```

Since every step is a legal move, the generated puzzle has a valid path back to the solved state.

#### 4. One thing you should check carefully

Make sure your scrambling code **doesn't accidentally call** `move()` **incorrectly**.

After generating a puzzle, check:

- Does the board contain every number exactly once?
- Is there exactly one `0` ?
- Is the board the correct size?
- Does every move actually move the blank?
- Does the game still recognize the solved board?

For a 4×4 puzzle, you should always have:

`</>` Plain text

```
0, 1, 2, 3, ... 15
```

exactly once.

---

#### 5. For your Lab 4 evidence

For Task 1, I would do this:

##### Before Task 1:

Show the original project generating a puzzle and, if possible, demonstrate the unsolvable-state problem.

##### After Task 1:

Run the modified project several times and show different starting boards.

You can also keep a screenshot/video of a generated puzzle and then solve it.

That gives you evidence that your change works.

**Don't move to Task 2 yet.** First test Task 1 properly. If you paste your **current `puzzle.py`** **after your changes**, I can check whether your Task 1 implementation actually guarantees solvability before you commit it.

ok task 1 is working. now tell me how to do task 2. it should detect when the puzzle is solved and stop the game properly. tell me

## exactly what to change

Yes. For **Task 2**, you should make a **small change in `game.py`** , not rewrite the project.

The requirement is:

Detect when the puzzle reaches the solved arrangement, end the game cleanly, and don't allow further commands after completion.

Your existing `puzzle.py` already has `solved()` , so **you can reuse it**.

### Step 1 — Open `game.py`

Find the `run()` method. It should have a loop roughly like this:

</> Python

```
while True:
 self.display()

 if self.puzzle.solved():
 print("Solved!")
 return

 command = input("Move (W/A/S/D, Q to quit): ").strip().lower()

 if command == "q":
 return

 if self.puzzle.move(command):
 self.moves += 1
```

If your code already looks like this, **Task 2 may already be partially implemented**.

### Step 2 — Make sure the solved check happens BEFORE input

The important order should be:

</> Plain text

```
display board
↓
check solved?
↓
YES → print message → stop
↓
NO
```



ask for W/A/S/D

So your `run()` should have the solved check **before**:

Python

```
command = input(...)
```

That prevents the program from asking for another command after the puzzle is solved.

### Step 3 — Use this structure

In `game.py`, make the `run()` method follow this pattern:

Python

```
def run(self):
 while True:
 self.display()

 if self.puzzle.solved():
 print("Solved!")
 print(f"Moves: {self.moves}")
 return

 command = input("Move (W/A/S/D, Q to quit): ").strip().lower()

 if command == "q":
 print("Game ended.")
 return

 if self.puzzle.move(command):
 self.moves += 1
```

**But don't blindly replace your whole `run()` method.** Keep your existing display/timer/input formatting and only make sure the solved check is in the correct place.

### Step 4 — Test it

You need to test **two things**.

#### Test A — Solved detection

Make the puzzle reach:

Plain text

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

You should see something like:

</> Plain text

```
Solved!
Moves: 25
```

and the program should **exit**.

It should NOT show:

</> Plain text

```
Move (W/A/S/D, Q to quit):
```

again.

### Test B — No commands after completion

After `Solved!` appears, type something like:

</> Plain text

```
w
```

It shouldn't do anything because the game has already ended.

---

## One important point for your lab

Since **Task 1 is already working**, don't modify `puzzle.py` unnecessarily for Task 2.

Your Task 2 change should mainly be:

</> Plain text

```
game.py
↓
check puzzle.solved()
↓
print completion message
```

↓

```
return from run()
```

Then test it, and **make a separate Git commit for Task 2**, because the lab specifically requires separate commits for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

If you paste your **current `game.py` after Task 1**, I can point out the **exact lines to add/change** rather than making you replace anything unnecessarily.

i changed task 2. how should i test it properly

Yes. Since **Task 2 is already changed**, test it specifically for the two requirements: **detect solved state** and **stop accepting commands afterward**.

## Test 1 — Normal gameplay still works

Run:

```
</> Bash
```

```
python main.py
```

Make a few valid moves.

Check that:

- The board changes correctly.
- The move count increases.
- The game doesn't stop before the puzzle is solved.

This confirms Task 2 didn't break normal gameplay.

---

## Test 2 — Check the solved state

The easiest way is to temporarily make the puzzle very easy to solve, or use a generated puzzle and solve it normally.

When the board reaches:

```
</> Plain text
```

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

you should get your completion message, for example:

</> Plain text

Solved!

and the program should **exit**.

The important thing is that it should **not ask for another move**.

✖ Wrong:

</> Plain text

Solved!

Move (W/A/S/D, Q to quit):

✔ Correct:

</> Plain text

Solved!

and then the program returns to the terminal.

---

### Test 3 — Make sure commands don't work after solving

This is the most important Task 2 test.

After you see:

</> Plain text

Solved!

try typing:

</> Plain text

w

or

</> Plain text

d

There should be **no new game action**, because the game has already ended.

---

## Test 4 — Make sure quitting still works

Start the game again:

</> Bash

```
python main.py
```

Immediately enter:

</> Plain text

q

The game should exit normally.

This makes sure your Task 2 change didn't break the existing quit functionality.

---

## Test 5 — Run it more than once

Run:

</> Bash

```
python main.py
```

several times.

Make sure:

</> Plain text

```
new game
↓
play
↓
```

```
solve
↓
Solved!
↓
program exits
```

happens consistently.

## What you should record for your lab

For **Task 2**, you don't necessarily need a separate video—the handout only requires the overall **before and after 10-second videos**.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

But keep a screenshot of the terminal showing:

 Plain text

```
Solved!
```

if you want evidence for your report/chat history.

## Before committing Task 2

Check these 4 things:

- ☐ Solved board is detected.
- ☐ Completion message is displayed.
- ☐ Game stops immediately after solving.
- ☐ Normal moves and `q` still work.

If all four work, **commit Task 2 separately** before starting Task 3. The lab specifically requires separate commits for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

now task 3. i need 3x3 4x4 and 5x5 modes. tell me what changes i have to make in main.py and game.py. dont change unnecessary things

Yes. For **Task 3**, keep the changes minimal. The requirement is only to allow the user to choose **3×3, 4×4, or 5×5**, and the existing game logic should work with the selected size.

You should mainly change **main.py** and **game.py**. `puzzle.py` already uses `self.size`, so your Task 1 scrambling and movement logic should automatically work for different sizes.

## 1. Change main.py

Currently you likely have:

`</>` Python

```
from game import SlidingPuzzle

if __name__ == "__main__":
 SlidingPuzzle().run()
```

Change it to:

`</>` Python

```
from game import SlidingPuzzle

if __name__ == "__main__":
 print("Choose puzzle size:")
 print("1. 3x3")
 print("2. 4x4")
 print("3. 5x5")

 choice = input("Enter your choice: ").strip()

 sizes = {
 "1": 3,
 "2": 4,
 "3": 5
 }

 if choice not in sizes:
 print("Invalid choice.")
 else:
 SlidingPuzzle(sizes[choice]).run()
```

**What this does**

If the user enters:

`</>` Plain text

1

it creates:

`</>` Plain text

3 × 3

If they enter:

</> Plain text

2

it creates:

</> Plain text

4 × 4

If they enter:

</> Plain text

3

it creates:

</> Plain text

5 × 5

---

## 2. Change `game.py`

Find your `SlidingPuzzle` constructor.

You probably currently have something like:

</> Python

```
def __init__(self):
 self.size = 4
 self.puzzle = Puzzle(self.size)
 self.moves = 0
 self.started = time.monotonic()
```

Change **only** the constructor parameter and the hardcoded size:

</> Python

```
def __init__(self, size):
 self.size = size
```

```
self.puzzle = Puzzle(self.size)
self.moves = 0
self.started = time.monotonic()
```

The important change is:

Python

```
def __init__(self, size):
```

and:

Python

```
self.size = size
```

instead of:

Python

```
self.size = 4
```

---

### 3. Don't change the rest of `game.py`

This is important.

You **don't need separate code** like:

Python

```
if size == 3:
 ...
elif size == 4:
 ...
elif size == 5:
 ...
```

Your existing code should already use `self.size` when displaying the board, checking the puzzle, etc.

So the flow becomes:

Plain text

```
main.py
 ↓
User chooses 3 / 4 / 5
 ↓
SlidingPuzzle(size)
 ↓
game.py
 ↓
Puzzle(size)
 ↓
puzzle.py
 ↓
3×3 / 4×4 / 5×5
```

---

## 4. One thing to check in `game.py`

If your display function has any **hardcoded 4**, don't leave it there.

For example, this is good:

Python

```
for row in self.puzzle.board:
 print(" ".join(f"{x:2}" for x in row))
```

But if you have something like:

Python

```
for i in range(4):
```

that would need to become:

Python

```
for i in range(self.size):
```

**Don't change anything unless you actually have a hardcoded 4.**

---

## 5. Test Task 3

Run:

`</> Bash`

```
python main.py
```

## Test 3×3

Choose:

`</> Plain text`

1

You should get a board containing:

`</> Plain text`

```
1 2 3
4 5 6
7 8 0
```

in some scrambled arrangement.

Check that:

- It is 3×3.
- Moves work.
- Timer works.
- Move count works.
- Solved detection works.

## Test 4×4

Run again:

`</> Bash`

```
python main.py
```

Choose:

`</> Plain text`

2

You should get:

</> Plain text

4 rows × 4 columns

## Test 5×5

Run again:

</> Bash

```
python main.py
```

Choose:

</> Plain text

3

You should get:

</> Plain text

5 rows × 5 columns

and the numbers should range from:

</> Plain text

1 ... 24

0

---

## One important test

Try an invalid choice:

</> Plain text

4

You should get:

</> Plain text

Invalid choice.

and the program should exit without starting a puzzle.

---

## Your Task 3 changes should basically be only this

**main.py :**

</> Plain text

```
hardcoded SlidingPuzzle()
 ↓
ask size
 ↓
SlidingPuzzle(selected_size)
```

**game.py :**

</> Plain text

```
def __init__(self):
 ↓
def __init__(self, size):

self.size = 4
 ↓
self.size = size
```

That's it. **Don't rewrite `puzzle.py` , don't change the movement logic, and don't change your Task 1 scrambling code.** Your existing `Puzzle` class is already designed around `self.size` , so this is the minimal change needed.

how do i test 3x3 4x4 and 5x5 after changing it and make sure moves and timer are working

Yes. Test **each size separately**, and for each one check **board size + moves + timer + solved detection**. This will give you confidence that Task 3 didn't break Task 1 or Task 2.

## 1. Test 3×3

Run:

`</> Bash`

```
python main.py
```

Choose:

`</> Plain text`

```
1
```

You should see a **3×3 board**, for example:

`</> Plain text`

```
2 5 1
7 0 3
8 4 6
```

## Check moves

Make one valid move.

For example, if `0` is in the middle:

`</> Plain text`

```
2 5 1
7 0 3
8 4 6
```

Press a key that moves a neighboring tile into the blank.

Check:

- The board changes.
- Only two positions change.
- Move count increases by 1.

Then make another valid move and check that the count increases again.

## Check timer

Before making a move, note the displayed time.

Wait about 5–10 seconds.

The timer should increase.

For example:

</> Plain text

Time: 00:05

then later:

</> Plain text

Time: 00:12

The timer should not reset after every move.

---

## 2. Test 4×4

Run again:

</> Bash

```
python main.py
```

Choose:

</> Plain text

2

You should get 4 rows × 4 columns:

</> Plain text

```
... ..
... ..
... ..
... ..
```

Check the same three things:

### Board

Exactly 16 positions.

Numbers should be:

</> Plain text

0-15

with one 0 .

## Moves

Make 2-3 valid moves.

Check:

</> Plain text

Moves: 1

Moves: 2

Moves: 3

or whatever your existing display format is.

## Timer

Wait a few seconds and make a move.

The timer should continue from where it was; it shouldn't restart.

---

## 3. Test 5×5

Run:

</> Bash

python main.py

Choose:

</> Plain text

3

You should get 5 rows × 5 columns.

There should be:

</> Plain text

25 positions

containing:

</> Plain text

0-24

with one 0 .

Again:

1. Make a valid move.
2. Check move count increases.
3. Wait a few seconds.
4. Check timer increases.
5. Make another move.
6. Check timer continues increasing.

## 4. Important: test invalid moves too

Since you already completed Task 2 and will later do Task 4, don't confuse the tests.

For Task 3, mainly make sure the **existing movement system works at all three sizes**.

For each size, try a move that is actually possible and confirm the board changes.

For example, if:

</> Plain text

1 2 3

4 0 5

6 7 8

pressing D /the appropriate valid direction should move 5 into the blank.

## 5. Quick testing table

You can literally use this checklist:

Test	3×3	4×4	5×5
Correct board size	✓	✓	✓
Correct numbers	✓	✓	✓
Exactly one blank	✓	✓	✓
Valid move works	✓	✓	✓
Move count increases	✓	✓	✓
Timer starts	✓	✓	✓
Timer keeps increasing	✓	✓	✓
Timer doesn't reset after move	✓	✓	✓
Solved detection still works	✓	✓	✓

If all of these pass, **Task 3 is working**.

### One especially important thing

Don't try to solve a randomly generated 5×5 manually just to test the timer. That's unnecessary. For Task 3, you only need to verify that the selected size is created correctly and that the existing game mechanics operate on that size.

Once you've tested all three sizes, **commit Task 3 separately**. Your lab instructions specifically require separate commits for each task.  [Lab\\_4\\_VibeCoding\\_Student\\_handout](#)

now task 4 says only successful slides should increase moves and invalid moves should not. also it should only show success when tile actually moves. what do i change in my code

Yes. For **Task 4**, the important thing is that your `puzzle.move()` should tell `game.py` whether a move was actually successful.

You should **not rewrite the project**. The cleanest fix is to use the return value of `move()` .

### 1. Check `puzzle.py`

Your `move()` function should return `True` **only when a tile actually moves**, and `False` when the requested move is invalid.

The structure should be like this:

`</>` Python

```
def move(self, direction):
 r, c = self.blank_pos()

 moves = {
 "w": (-1, 0),
 "s": (1, 0),
 "a": (0, -1),
 "d": (0, 1)
 }

 if direction not in moves:
 return False

 dr, dc = moves[direction]
 nr, nc = r + dr, c + dc

 if not (0 <= nr < self.size and 0 <= nc < self.size):
 return False

 self.board[r][c], self.board[nr][nc] = \
```

The important parts are:

`</>` Python

```
return False
```

when the move is impossible, and:

`</>` Python

```
return True
```

after the swap actually happens.

## 2. Then check `game.py`

Your game loop should use that return value.

You want this:

</> Python

```
if self.puzzle.move(command):
 self.moves += 1
 print("Move successful!")
```

NOT this:

</> Python

```
self.puzzle.move(command)
self.moves += 1
print("Move successful!")
```

The second version would count and report **invalid moves too**.

---

### 3. What happens now?

Suppose the blank is at the top:

</> Plain text

```
0 2 3
1 4 5
6 7 8
```

If you try to move up:

</> Plain text

```
W
```

that's impossible.

So:

</> Plain text

```
move() → False
```

Therefore:

</> Plain text

```
moves → does NOT increase
"Move successful!" → NOT printed
```

But if you press a valid direction:

</> Plain text

```
s
```

then:

</> Plain text

```
move() → True
```

and:

</> Plain text

```
moves → +1
"Move successful!" → printed
```

---

## 4. One more thing to check

If your `game.py` currently has something like:

</> Python

```
if self.puzzle.move(command):
 self.moves += 1

print("Move successful!")
```

the `print()` is **outside** the `if` .

Move it inside:

</> Python

```
if self.puzzle.move(command):
 self.moves += 1
```

```
print("Move successful!")
```

That's a very common cause of this Task 4 bug.

**So your final logic should be**

</> Plain text

User enters command



puzzle.move()



True



tile moves



moves + 1



success

False



nothing happens



moves unchanged



no success message

**Don't change anything else.** Your Task 1 scrambling, Task 2 solved detection, and Task 3 size selection can remain as they are.

how do i test task 4 properly like one valid move and one invalid move

Yes. For Task 4, do a very simple **valid-move vs invalid-move test**. You want to prove three things:

1. Valid move → tile moves.
2. Valid move → move count increases by 1 and success is shown.
3. Invalid move → board doesn't change, move count doesn't increase, and **no success message** appears.

## Step 1 — Start the game

</> Bash

```
python main.py
```

Choose any size, preferably 3×3 because it's easier to see.

For example, suppose you get:

</> Plain text

```
2 0 3
1 4 6
7 5 8
```

Assume the current move count is:

</> Plain text

Moves: 0

---

## Step 2 — Test a VALID move

The blank 0 is in the middle.

Choose a direction that moves a neighboring tile into the blank.

For example, if S moves the tile below into the blank, press:

</> Plain text

S

The board should change:

</> Plain text

```
2 5 3
1 0 6
7 4 8
```

You should see:

</> Plain text

Moves: 1

and your success message, if your project displays one.

✅ Valid move passed.

---

## Step 3 — Now test an INVALID move

Look at the new board:

</> Plain text

```
2 5 3
1 0 6
7 4 8
```

The blank is still in the middle.

Now choose a direction where there is **no tile** to move.

For example, if the blank is at the middle and you move in a direction toward the outside edge, choose the appropriate invalid direction based on your project's W/A/S/D mapping.

The key is: **choose a direction that would move the blank outside the board.**

When you press it:

**The board should stay exactly the same**

</> Plain text

```
2 5 3
1 0 6
7 4 8
```

**Move count should stay:**

</> Plain text

Moves: 1

NOT:

</> Plain text

Moves: 2

And importantly, you should **not** see:

</> Plain text

Move successful!

for the invalid move.

### What you're looking for

Action	Board	Move count	Success message
Valid move	Changes	+1	✔ Yes
Invalid move	Same	No change	✘ No

### Best way to demonstrate Task 4

For your testing evidence, capture this sequence:

```
</> Plain text

Initial:
Moves: 0

↓ valid move

Board changes
Moves: 1
Move successful!

↓ invalid move

Board unchanged
Moves: 1
(no success message)
```

That directly demonstrates the requirement from Task 4.

**One caution:** your project's exact W/A/S/D direction mapping matters. If you send me the current `move()` function from your `puzzle.py` , I can tell you **exactly which key will be valid and which key will be invalid for the board you get**, so you don't waste time testing the wrong direction.

i finished all 4 tasks. give me a final checklist to check before submitting so i dont miss anything

Absolutely. Since you finished all 4 tasks, use this **final checklist in order** before submitting. The Lab 4 handout requires the before/after videos, updated code, exported chat history, and separate commits for each task.

 Lab\_4\_VibeCoding\_Student\_handout

## ✓ Lab 4 Final Submission Checklist

### 1. Test the complete project one final time

Run:

⌕ Bash

```
python main.py
```

Check:

- ☐ Program starts without errors.
- ☐ 3×3 option works.
- ☐ 4×4 option works.
- ☐ 5×5 option works.
- ☐ Board contains the correct numbers.
- ☐ Exactly one blank ( 0 ) exists.
- ☐ Valid moves work.
- ☐ Move counter works.
- ☐ Timer works.
- ☐ Invalid moves don't increase the move count.
- ☐ Invalid moves don't show a success message.
- ☐ Solved puzzle is detected.
- ☐ Game stops after the puzzle is solved.
- ☐ q still quits the game properly.

---

## 2. Specifically verify each task

### Task 1 — Solvable puzzles

Run the game several times.

⌕ Plain text

```
Run → generate puzzle
```

```
Run → generate another puzzle
```

```
Run → generate another puzzle
```

Check:

- ☐ Different puzzles are generated.
- ☐ Every generated puzzle is solvable.

- ☐ Your original `random.shuffle()` problem has been fixed.
- ☐ You didn't unnecessarily change the rest of the puzzle logic.

## Task 2 — Solved state

Check:

`</>` Plain text

Solved board

↓

" Solved! "

↓

Game exits

- ☐ Solved state is detected.
- ☐ Completion message appears.
- ☐ Game doesn't ask for another move.
- ☐ Commands aren't processed after completion.

## Task 3 — Three sizes

Test all three:

`</>` Plain text

1 → 3×3

2 → 4×4

3 → 5×5

- ☐ 3×3 works.
- ☐ 4×4 works.
- ☐ 5×5 works.
- ☐ Moves work in all three.
- ☐ Timer works in all three.
- ☐ Solved detection still works.
- ☐ Invalid size input is handled properly.

## Task 4 — Valid vs invalid moves

Do one of each:

`</>` Plain text

VALID:

tile moves → Moves +1 → success message

INVALID:

tile doesn't move → Moves unchanged → NO success message

- ☐ Valid move increases counter.
  - ☐ Invalid move does NOT increase counter.
  - ☐ Success message appears only for valid moves.
  - ☐ Invalid move doesn't change the board.
- 

### 3. Check your Git commits

This is **very important** because the handout specifically says:

"Have Separate Commits for each task."



Lab\_4\_VibeCoding\_Student\_handout

Run:

 Bash

```
git log --oneline
```

You should have separate commits corresponding to:

 Plain text

Task 1

Task 2

Task 3

Task 4

For example:

 Plain text

```
abc1234 Task 4 - fix invalid move handling
```

```
def5678 Task 3 - add puzzle size selection
```

```
ghi9012 Task 2 - handle solved puzzle
```

```
jkl3456 Task 1 - generate solvable puzzles
```

The exact wording doesn't matter as much as having **separate commits**.

---

## 4. Check your GitHub repository

Push your final code:

 Bash

```
git status
```

Make sure there are no unexpected changes.

Then:

 Bash

```
git push
```

Open your GitHub repository in the browser and check:

- ☐ Latest code is visible.
- ☐ `main.py` is updated.
- ☐ `game.py` is updated.
- ☐ `puzzle.py` is updated.
- ☐ `README.md` is still there.
- ☐ `requirements.txt` is still there.
- ☐ No unnecessary files were added.
- ☐ You did **not** create a PR to `SETAPESU26` .

The handout specifically says not to raise a PR to the main `SETAPESU26` repository.

 Lab\_4\_VibeCoding\_Student\_handout

---

## 5. BEFORE video

The instructions require a **10-second video before making changes.**  Lab\_4\_VibeCoding\_Student\_handout

Make sure you have:

 Plain text

```
BEFORE.mp4
```

or similar.

It should show the **original project running before your modifications.**

## 6. AFTER video

The instructions require another **10-second video after all changes.**



Lab\_4\_VibeCoding\_Student\_handout

Your after video should preferably show the **final working project.**

If possible, show:

 Plain text

Choose puzzle size

1. 3x3
2. 4x4
3. 5x5

and some gameplay.

## 7. Chat history

This is easy to forget.

The handout explicitly requires:

"Chat history exported as a doc/pdf."



Lab\_4\_VibeCoding\_Student\_handout

So make sure you have:

 Plain text

Chat\_History.pdf

It should contain your Vibe Coding conversation/prompts showing how you worked through the tasks.

**Don't delete the prompts you used.**

## 8. Your Lab-4 folder

Your final submission should contain the required items:

 Plain text

```
Lab-4/
|
├─ BEFORE.mp4
|
├─ AFTER.mp4
|
├─ Chat_History.pdf
|
└─ Updated_Code/
 ├── main.py
 ├── game.py
 ├── puzzle.py
 ├── README.md
 └─ requirements.txt
```

The exact folder arrangement may depend on how your lab repository is organized, but the handout requires **videos**, **updated code**, and **exported chat history** under the **Lab-4** folder.

 Lab\_4\_VibeCoding\_Student\_handout



## Final 10-second check

Before you submit, ask yourself:

### Plain text

- ☐ Task 1 – Every generated puzzle is solvable
- ☐ Task 2 – Solved puzzle stops correctly
- ☐ Task 3 – 3×3 / 4×4 / 5×5 work
- ☐ Task 4 – Only successful moves increase count
- ☐ Valid move → success message
- ☐ Invalid move → no success message
- ☐ Timer works
- ☐ Before video exists
- ☐ After video exists
- ☐ Chat history PDF exists
- ☐ Task 1 separate commit
- ☐ Task 2 separate commit
- ☐ Task 3 separate commit
- ☐ Task 4 separate commit
- ☐ Final code pushed to your repo
- ☐ No PR to SETAPESU26
- ☐ Lab-4 folder contains everything required

**If every box is checked, you're ready to submit.**

my mam will ask viva on this project. tell me what questions she can ask from this project and the changes i made. give simple answers which i can understand and explain myself

Yes. Since your mam is likely to ask based on **your actual project + the 4 changes you made**, prepare these first. I'll keep the answers simple enough that you can explain them in your own words.

## Sliding Puzzle Lab 4 — Viva Questions

### A. Basic project questions

#### 1. What is your project?

**Answer:**

It is a Python-based sliding puzzle game. The player moves numbered tiles into the blank space using keyboard controls and tries to arrange the tiles in the correct order.

---

#### 2. How does the sliding puzzle work?

**Answer:**

There is one empty space represented by `0`. A tile next to the empty space can move into it. By repeatedly moving tiles, we try to reach the solved arrangement.

---

#### 3. What is the goal of the game?

**Answer:**

The goal is to arrange all the tiles in increasing order, with the blank space at the end.

For  $3 \times 3$ :

 Plain text

```
1 2 3
4 5 6
7 8 0
```

---

#### 4. What language did you use?

**Answer:**

Python.

---

## 5. How do you run the project?

Answer:

```
</> Bash
```

```
python main.py
```

`main.py` starts the game.

---

## File-related questions

### 6. What is the role of `main.py` ?

Answer:

`main.py` is the entry point. It starts the game and, after my Task 3 change, asks the user to select 3×3, 4×4, or 5×5.

---

### 7. What is the role of `game.py` ?

Answer:

`game.py` controls the gameplay. It handles things like displaying the game, taking user input, counting moves, timing, and checking whether the puzzle is solved.

---

### 8. What is the role of `puzzle.py` ?

Answer:

`puzzle.py` contains the actual puzzle logic. It creates the board, finds the blank space, moves tiles, scrambles the puzzle, and checks whether the puzzle is solved.

---

### 9. What is the role of `requirements.txt` ?

Answer:

It contains the project's Python dependency information. This project mainly uses Python's standard library, so there are no major external packages required.

---

### 10. Why is the project divided into different files?

**Answer:**

To keep the code modular. The game control and puzzle logic are separated, so it is easier to understand and modify the project.

---

## Task 1 — Solvable puzzle

This is **very important** for your viva.

### 11. What was the problem in the original code?

**Answer:**

The original code used `random.shuffle()` to randomly arrange the tiles. A completely random arrangement is not always solvable in a sliding puzzle.

---

### 12. Why is `random.shuffle()` a problem?

**Answer:**

Because it can produce a board configuration that cannot be reached from the solved state using legal moves.

---

### 13. How did you fix it?

**Answer:**

I start with a solved puzzle and then make a series of random **legal moves**. Since every move is legal, the final scrambled board can always be reached from the solved board, so it is solvable.

---

### 14. Why does your new method guarantee solvability?

**Answer:**

Because I don't create a completely random arrangement. I start from the solved state and only perform valid moves. So there is always a path back to the solved state by reversing those moves.

This is a **very good answer to remember**.

---

### 15. Did you change the whole project for Task 1?

**Answer:**

No. I only changed the puzzle generation logic. I kept the existing movement and game structure.

---

## 16. How did you test Task 1?

### Answer:

I generated the puzzle multiple times and checked that different starting states were produced and that they could be solved. I also checked that the board still contained all required tiles and one blank space.

---

## Task 2 — Solved state

## 17. How does the program know that the puzzle is solved?

### Answer:

The `solved()` function checks whether the board is in the correct final arrangement.

---

## 18. What is the solved state for a 4×4 puzzle?

### Answer:

 Plain text

```
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0
```

---

## 19. What did you change for Task 2?

### Answer:

I made sure the game checks `puzzle.solved()` during the game loop. When it becomes true, it displays the completion message and returns from the game instead of asking for another move.

---

## 20. Why should the game stop after solving?

### Answer:

Because the game is already completed. Allowing more commands after that would be unnecessary and could change the solved board.

---

## 21. What happens after `return` in the game loop?

**Answer:**

It exits the `run()` function, so the game stops.

---

## Task 3 — 3×3, 4×4, 5×5

**22. What was the problem before Task 3?**

**Answer:**

The original game was fixed to a 4×4 puzzle.

---

**23. What did you add?**

**Answer:**

I added a size selection in `main.py`, so the user can choose:

 Plain text

```
1 → 3×3
2 → 4×4
3 → 5×5
```

---

**24. What did you change in `game.py` ?**

**Answer:**

Instead of hardcoding:

 Python

```
self.size = 4
```

I made the size a parameter and used the selected size.

For example:

 Python

```
def __init__(self, size):
 self.size = size
```

---

## 25. Why did you pass `size` to `Puzzle()` ?

**Answer:**

Because the puzzle needs to know the board dimensions. So the selected size is passed to the `Puzzle` class.

---

## 26. Did you create separate logic for 3×3, 4×4 and 5×5?

**Answer:**

No. I reused the existing general puzzle logic. The board size is passed as a parameter, so the same code works for all three sizes.

This is a strong answer.

---

## 27. Why is this better than writing separate code for each size?

**Answer:**

It avoids duplicate code and makes the program easier to maintain. The same logic can work with different sizes.

---

## 28. How did you test Task 3?

**Answer:**

I ran the program separately with 3×3, 4×4 and 5×5 and checked the board size, tile movement, move counter, timer and solved detection.

---

## Task 4 — Valid and invalid moves

This is another very likely viva question.

## 29. What was the problem with invalid moves?

**Answer:**

An invalid move should not be counted as a move because no tile actually moved. It also should not show a successful-move message.

---

## 30. What is a valid move?

**Answer:**

A valid move is when a tile can actually move into the blank space.

### 31. What is an invalid move?

**Answer:**

An invalid move is when the requested direction would move the blank outside the board, or the command isn't a valid movement.

---

### 32. How did you fix Task 4?

**Answer:**

I made `puzzle.move()` return whether the move was successful.

⟨/⟩ Plain text

```
True → tile moved
False → tile didn't move
```

Then `game.py` increases the move count and displays the success message only when the result is `True`.

---

### 33. Why use `True` and `False` ?

**Answer:**

It gives the game a simple way to know whether the movement actually happened.

---

### 34. What happens for a valid move?

**Answer:**

⟨/⟩ Plain text

```
move() → True
 ↓
move count +1
 ↓
success message
```

### 35. What happens for an invalid move?

**Answer:**

 Plain text

```
move() → False
 ↓
move count unchanged
 ↓
no success message
```

---

### 36. Why shouldn't an invalid move increase the counter?

**Answer:**

Because the player hasn't actually moved a tile. The counter should represent successful tile movements.

---



## Input and movement questions

### 37. What do W, A, S and D do?

**Answer:**

They control the movement direction of the blank/tile according to the project's movement mapping.

---

### 38. How does the program know where the blank is?

**Answer:**

The `blank_pos()` function searches the board for the value `0` and returns its row and column.

---

### 39. Why is the blank represented by `0` ?

**Answer:**

`0` is used as a special value to represent the empty position on the board.

---

### 40. How does a tile actually move?

**Answer:**

The program swaps the blank position with the neighboring tile. So the tile moves into the blank and the blank moves to the tile's old position.

---



## Timer and moves

### 41. How is the move count maintained?

Answer:

There is a `moves` variable in `game.py`. It increases only when `puzzle.move()` reports a successful move.

---

### 42. What happens to the move count for an invalid move?

Answer:

It stays unchanged.

---

### 43. How does the timer work?

Answer:

The game records the starting time and uses the elapsed time to display how long the player has been playing.

---

### 44. Does changing from 4×4 to 5×5 require a completely different timer?

Answer:

No. The same timer logic can be reused because the timer is independent of the board size.

---



## Questions about your overall changes

### 45. What were the four tasks you completed?

Answer:

</> Plain text

Task 1 → Make every generated puzzle solvable

Task 2 → Detect solved state and stop the game

Task 3 → Add 3×3, 4×4 and 5×5 modes

Task 4 → Count and report only successful moves

Memorize this one.

---

## 46. Which files did you modify?

**Answer:**

I modified the files needed for the four tasks, mainly `puzzle.py` , `game.py` , and `main.py` .

---

## 47. Why didn't you rewrite the whole project?

**Answer:**

Because the existing project already had most of the required functionality. I only changed the parts required for each task and reused the existing code.

---

## 48. How did you test your changes?

**Answer:**

I tested each task separately and then tested the complete project. I checked different puzzle sizes, valid and invalid moves, the timer, move count, solvability and solved-state detection.

---



## Vibe Coding questions

Your lab is specifically about **Vibe Coding**, so your mam may ask these.

## 49. What is Vibe Coding?

**Answer:**

It is using an AI coding assistant through natural-language prompts to help understand, modify or create code, while the developer still tests and verifies the code.

---

## 50. Did you blindly accept the AI-generated code?

**Answer:**

No. I checked what the code was doing, tested the changes and verified that the requirements were satisfied.

---

## 51. How did you make sure the AI didn't break the existing project?

**Answer:**

I made small changes task by task and tested the project after each change instead of replacing the whole project.

---

## 52. Why did you make separate commits?

**Answer:**

So each task has its own change history. If something goes wrong, it is easier to identify which task introduced the problem.

---

## ★ If your mam asks "Explain your project changes"

Give this answer:

"Initially, the project was a 4×4 sliding puzzle. The first problem was that `random.shuffle()` could generate an unsolvable puzzle, so I changed the generation to start from the solved state and scramble it using legal moves. Then I made sure the game detects the solved state and stops. After that, I added size selection for 3×3, 4×4 and 5×5 by passing the selected size to the game and puzzle. Finally, I changed the movement handling so that only a successful tile movement increases the move count and displays the success message. I tested each task separately and then tested the complete project."

That's a very good 1-minute viva answer.

## 🔥 10 questions I would memorize first

If you don't have much time, memorize these:

1. What is your project?
2. What was wrong with `random.shuffle()` ?
3. How did you guarantee solvability?
4. How do you detect the solved state?
5. What did you change for 3×3/4×4/5×5?
6. Why did you pass `size` as a parameter?
7. What is a valid vs invalid move?
8. How does `True/False` from `move()` help?
9. Why doesn't an invalid move increase the counter?
10. What are the four tasks you completed?

If you can answer those **without looking at notes**, you'll be in a good position for the viva.